# Software Security Standard - (SSS)

## Software Security Standard - (SSS)

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Category: Governance & Enforcement

Subcategory: Software Lifecycle Governance

Type: Parent Standard

Governed Space: Software Security Governance

Standard ID: SLGA-SSS-006

Origin ID: OOF.PS.SLGA.06.01

Version: 1.0

Status: Canonical · Open Standard

Effective Date: 13.10.2026

Compatibility: OOF® Methodology OS™ · Modular OOF® Architecture Ecosystem

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: MIP® — Methodological Intellectual Property

Canonical Language: English (UCL™)

## Canonical Definition

## The Software Security Standard (SSS) governs the establishment,

assessment, verification, determination, acceptance, and lifecycle continuity of software security conditions.

## SSS establishes the methodological conditions under which the security of a

software artifact, component, application, service, or integrated system can be recognized as an identifiable, evidence-supported, context-dependent governance state rather than an assumption derived from implemented controls, successful security scans, the absence of reported vulnerabilities, or declared compliance.

## It governs the relationships among software security requirements,

protected assets, threat conditions, trust boundaries, attack surfaces, security exposure, protective controls, security testing, vulnerability identification, remediation verification, residual security risk, and security conformity and acceptance.

## SSS distinguishes declared security from demonstrated security, identified

threats from actual exposure, implemented controls from verified effectiveness, vulnerability closure from verified remediation, and security conformity from authorized acceptance of residual risk.

## It applies to software created, modified, integrated, maintained, tested,

assessed, or operated by humans, organizations, automated development systems, AI coding assistants, autonomous software agents, and software infrastructure mechanisms.

SSS is technology-independent and does not prescribe a particular programming language, security tool, development methodology, security certification scheme, or deployment platform.

Its principal output is:

## Governed Software Security State

Its foundational distinction is:

Declared Software Security ≠ Demonstrated Software Security.

## Operational Role

The operational role of SSS is:

To ensure that software security requirements, relevant threats and exposure conditions, protective controls, verification evidence, vulnerabilities, remediation states, residual risks, and security acceptance decisions remain identifiable, contextually applicable, technically assessable, evidence-supported, appropriately authorized, and traceable to the actual software artifact and configuration throughout its lifecycle.

SSS provides the security-specific governance layer within SLGA™.

It establishes the conditions under which software security may be competently evaluated and recognized.

It does not independently authorize software creation, configuration changes, release, deployment, incident-response operations, or legal compliance certification.

## Core Question

How can software security be defined, demonstrated, assessed, and accepted through competent evidence while ensuring that implemented controls, successful scans, closed vulnerability records, or declared compliance are not mistaken for proof that software is sufficiently protected against relevant threats under its actual operating conditions?

## Position in Architecture Governance Flow

The SLGA™ governance flow is:

## Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting:

## Software Lifecycle Control

SSS governs the sixth lifecycle domain:

## Software Security

Its upstream governance references include:

• Software Requirements Standard (SRS): Governed software requirements

• Software Creation Standard (SCS): Governed software creation and

• Software Dependency Standard (SDS): Governed software dependencies

• Software Configuration Standard (SCFS): Governed software

• Software Quality Standard (SQS): Governed software quality

artifact provenance. configuration.

Its principal output is:

## Governed Software Security State

Downstream consumers may include:

## - Software Release Standard (SRLS)

- Software Deployment Standard (SDPS) - Software Change Management Standard (SCMS) - Software Retirement Standard (SRTS) - Software Lifecycle Control Standard (SLCS)

## The lifecycle sequence establishes governance dependencies, not a

restriction requiring security work to occur only after quality assessment.

## Security requirements, threat analysis, control verification, and

vulnerability treatment may operate throughout the software lifecycle.

• Governed Space

## Software Security Governance

The governed space includes:

• Software security object identity

• Security purpose and scope

• Artifact and configuration correspondence

• Security requirements and protective objectives

• Security properties

• Protected assets

• Security assumptions

• Trust boundaries

• Threat identification and applicability

• Attack surfaces and exposure

• Security control definition and applicability

• Security control implementation and effectiveness

• Authentication and authorization security

• Data and interface protection

• Dependency and configuration security

• Software supply-chain security evidence

• Secrets protection

• Security testing and coverage

• Security evidence competence and integrity

• Vulnerability identification and confirmation

• Vulnerability classification, severity, exploitability, exposure, and

• Vulnerability treatment prioritization

• Remediation and remediation verification

• Security regression

• Residual security risk

• Security exceptions

• Security conformity

• Security acceptance authority

• Conditional security acceptance

• Security uncertainty and conflicts

• Security evidence gaps

• Critical security gaps

• Security continuity and reassessment

• Governed software security states

impact.

## Software Security Governance Object

## Every material software security determination shall reference an

identifiable Software Security Governance Object.

The object shall preserve, where applicable:

Security Object Identity: The governed software security subject.

## Software Object: The component, application, service, or integrated

system.

Artifact Identity: The software artifact actually assessed.

## Configuration Reference: The relevant configuration and execution

conditions.

Security Purpose: The protective objectives.

Security Scope: The boundaries of the determination.

Requirements: Applicable software security requirements.

Threat Context: Relevant threats and exposure conditions.

Controls: Applicable protective mechanisms.

Evidence: Verification and assessment records.

Vulnerabilities: Known or suspected security weaknesses.

Residual Risk: Remaining security exposure or uncertainty.

Acceptance: Any competent security acceptance decision.

Governance State: The recognized software security condition.

## A software security governance object shall not be replaced by a generic

statement that a product is secure.

Security Declaration ≠ Governed Security Object.

## Software Security Governance Principles

SSS establishes the following principles:

Identifiable Security Subject: Every material determination must concern identifiable software.

## Contextual Applicability: Security conditions depend on software purpose,

exposure, configuration, and intended operation.

Explicit Protective Requirements: Material protection objectives must remain identifiable.

Threat Correspondence: Controls and assessments must correspond to relevant threat conditions.

## Evidence-Based Determination: Security conclusions require competent

supporting evidence.

Verified Control Effectiveness: Control implementation is not proof of protection.

## Vulnerability Transparency: Known vulnerabilities and unresolved findings

remain visible.

## Bounded Assurance: Security claims cannot exceed their supporting

evidence.

## Legitimate Acceptance Authority: Security assessment and acceptance of

residual risk remain distinct.

Lifecycle Continuity: Security determinations must survive as traceable records across changes.

## Material Reassessment: Changes in threats, software, configuration, or

exposure may invalidate previous determinations.

## Non-Absolute Security: No finite assessment demonstrates invulnerability

against every possible attack.

## Software Security Purpose

SSS shall establish the security purpose relevant to the governed software.

Purposes may include:

• Preserving confidentiality

• Protecting information integrity

• Supporting service availability

• Preventing unauthorized actions

• Protecting privileged operations

• Preventing unauthorized code or configuration modification

• Preserving transaction integrity

• Protecting communication between software components

• Limiting compromise propagation

• Supporting secure recovery

## The purpose shall remain connected to actual software functions and

protected assets.

Security Objective ≠ Implemented Security Mechanism.

## Software Security Scope

SSS shall establish the boundaries of each material security assessment.

Scope may concern source code, executable artifacts, libraries, services, interfaces, integrated systems, configurations, dependencies, and operating environments.

The assessment shall identify relevant inclusions and exclusions.

## A component assessment shall not automatically establish security of the

integrated application.

Component Security Assessment ≠ System Security Assurance.

## Software Artifact Identity

## SSS shall preserve the identity of the artifact to which a security

determination applies.

Relevant references may include:

• Artifact identifier

• Version

• Build identity

• Content integrity reference

• Dependency composition

• Configuration baseline

• Assessment environment

## A security report concerning one build shall not automatically establish

the security condition of another build.

Security Assessment of Version A ≠ Security Assessment of Version B.

## Software Configuration Correspondence

## SSS shall preserve the relationship between security evidence and relevant

configuration conditions.

These may include:

• Enabled features

• Authentication settings

• Authorization configuration

• Network exposure

• Debug interfaces

• Cryptographic settings

• Runtime privileges

• Dependency versions

• External integrations

A correctly implemented control may be ineffective when disabled or misconfigured.

Security Control Presence ≠ Active Security Protection.

## Security Requirement Governance

## SSS shall establish identifiable security requirements relevant to software

behavior and protection.

Requirements may concern authentication, authorization, data protection, input handling, secure communications, integrity verification, resilience, security logging, and prevention of unauthorized execution.

Each material requirement should identify its purpose, applicability, scope, relevant software object, and verification expectations.

Defined Security Requirement ≠ Demonstrated Security.

## Security Requirement Applicability

## SSS shall determine which security requirements apply to the actual

software context.

Applicability may depend on protected assets, intended use, user privileges, integration relationships, deployment conditions, and relevant threats.

Requirements shall not be treated as universally applicable without context.

General Security Requirement ≠ Applicable Security Requirement.

## Security Requirement Traceability

## SSS shall preserve traceability between requirements, protective controls,

verification evidence, vulnerabilities, and security determinations.

The reference relationship is:

## Security Requirement → Relevant Threat or Property → Protective Control →

Verification → Evidence → Security Determination

Traceability alone does not establish satisfaction.

Traceable Security Requirement ≠ Verified Security Requirement.

## Security Assumption Governance

## SSS shall identify material assumptions on which security conclusions

depend.

Examples include:

• Trusted network segmentation

• Correct identity-provider behavior

• Protected signing keys

• Secure infrastructure configuration

• Trusted external services

• Limited administrative access

• Correct dependency behavior

Assumptions must remain distinguishable from verified conditions.

Where a material assumption is invalidated, affected security conclusions shall be reassessed.

Assumed Protection ≠ Demonstrated Protection.

## Protected Asset Identification

SSS shall identify assets requiring protection.

Assets may include:

• Personal information

• Financial records

• Authentication credentials

• Cryptographic keys

• Operational commands

• Business-sensitive information

• Software integrity

• Service availability

• Critical application functions

## Asset identification provides the basis for evaluating security

consequences.

Protected Asset Importance ≠ Vulnerability Severity.

## Security Property Governance

SSS shall establish the relevant security properties of the software.

These may include:

• Confidentiality

• Integrity

• Availability

• Authenticity

• Authorization correctness

• Isolation

• Resistance to unauthorized modification

• Recovery integrity

A property shall be associated with assessable conditions where technically feasible.

Declared Security Property ≠ Verified Security Property.

## Trust Boundary Governance

SSS shall identify material trust boundaries within and around the software.

## Examples include boundaries between users and applications, services and

databases, internal and external APIs, development and production systems, and autonomous agents and accessible tools.

## A trust boundary shall remain associated with relevant authentication,

authorization, data handling, and control requirements.

Technical Access Across a Boundary ≠ Legitimate Authority.

## Threat Identification

SSS shall recognize identifiable threat conditions relevant to the software.

Threats may include:

• Unauthorized access

• Credential compromise

• Injection attacks

• Privilege escalation

• Data manipulation

• Service disruption

• Malicious dependency behavior

• Unauthorized software modification

• Abuse of privileged interfaces

Identifying a possible threat does not establish that the threat has been realized.

Identified Threat ≠ Observed Attack.

## Threat Applicability

SSS shall assess whether a threat is relevant to the software object, configuration, exposure, and operating environment.

Threat applicability may depend on attack-surface accessibility, privileges, dependencies, protected assets, and trust boundaries.

## A theoretical threat pattern shall not automatically be represented as an

actual software exposure.

Known Threat Pattern ≠ Demonstrated Exposure.

## Threat Modeling Governance

SSS shall govern the evidentiary use of threat models.

A material threat model should identify:

• Software scope

• Protected assets

• Trust boundaries

• Relevant assumptions

• Threat scenarios

• Potential attack paths

• Applicable controls

• Known limitations

Threat modeling supports security analysis but does not substitute for verification.

Completed Threat Model ≠ Demonstrated Software Security.

## Attack Surface Identification

SSS shall identify relevant points through which software may be accessed, influenced, or attacked.

Attack surfaces may include:

• Public APIs

• Authentication endpoints

• Administrative interfaces

• File-processing functions

• Network services

• Update mechanisms

• External integrations

• Agent-accessible tools

• Plugin interfaces

An attack surface is not automatically a vulnerability.

Attack Surface ≠ Confirmed Vulnerability.

## Security Exposure Assessment

SSS shall assess the conditions under which relevant threats may reach the software.

Exposure may depend on:

• Network accessibility

• User privileges

• Enabled functionality

• Configuration

• Deployment environment

• External dependencies

• Available mitigations

## Exposure shall remain distinguishable from exploitability and observed

exploitation.

Security Exposure ≠ Demonstrated Exploitation.

## Security Control Definition

## SSS shall establish identifiable protective controls corresponding to

applicable security requirements and threats.

Controls may include authentication, authorization, input validation, output encoding, cryptographic protection, isolation, integrity verification, rate limiting, secure configuration, and security logging.

Controls shall remain connected to the protective purpose they are intended to serve.

Defined Security Control ≠ Implemented Security Control.

## Security Control Applicability

SSS shall determine whether a selected control is suitable for the relevant security conditions.

## Applicability depends on the protected asset, threat model, software

architecture, trust boundary, and operating environment.

A generally recommended control may be insufficient for a particular risk.

Recommended Control ≠ Sufficient Protection.

## Security Control Implementation

SSS shall preserve evidence that the required control exists in the relevant software artifact or configuration.

Evidence may include code references, configuration records, build references, deployment settings, and integration records.

Implementation evidence shall not be treated as proof of effectiveness.

Implemented Security Control ≠ Verified Security Control.

## Security Control Effectiveness

## SSS shall assess whether a control provides the intended protection under

relevant conditions.

Verification should consider:

• Applicable threats

• Correct artifact

• Relevant configuration

• Suitable assessment method

• Expected protective behavior

• Observed protective behavior

• Known limitations

A control that executes successfully may still fail to prevent the relevant attack.

Successful Control Execution ≠ Demonstrated Protective Effectiveness.

## Security Architecture Correspondence

## SSS shall preserve correspondence between documented security architecture

and the implemented software.

## Relevant relationships include component boundaries, authentication paths,

authorization checks, data flows, dependency interactions, and external interfaces.

## Architectural documentation shall not be treated as independent evidence

that the implemented system conforms.

Documented Security Architecture ≠ Verified Security Implementation.

## Authentication Security

## SSS shall govern the security assessment of software mechanisms used to

establish and verify identity.

## Relevant conditions include credential validation, session handling, token

verification, authentication failure behavior, and identity-provider integration.

## Successful authentication shall not automatically authorize subsequent

actions.

Authenticated Identity ≠ Authorized Action.

## Authorization Security

SSS shall assess whether software operations are constrained by applicable permissions.

This includes:

• Resource access

• Role boundaries

• Delegated permissions

• Privileged functions

• Agent tool access

• Data access

• Constituent action authorization

Authority for an objective does not automatically authorize every action used to achieve it.

Task Authority ≠ Constituent Action Authority.

SSS governs technical enforcement and verification.

## General legitimacy of authority remains governed by competent

authority-governance mechanisms.

## Data Protection Security

## SSS shall assess security conditions protecting data processed,

transmitted, or stored by software.

## Relevant conditions include access restrictions, confidentiality,

integrity, secure transmission, secure storage, and prevention of unintended disclosure.

The existence of encryption does not establish comprehensive data protection.

Encrypted Data ≠ Fully Protected Data.

## Input and Output Security

SSS shall govern security-relevant handling of inputs and outputs.

Relevant conditions may include input validation, parsing, injection resistance, output encoding, file handling, command construction, and error disclosure.

## Successful processing of valid inputs shall not establish resistance to

malicious inputs.

Functional Input Handling ≠ Secure Input Handling.

## Software Interface Security

SSS shall assess security conditions governing software interfaces.

Interfaces may include APIs, administrative endpoints, plugin interfaces, service connections, and autonomous agent tools.

The ability to exchange information or execute operations does not establish that the interface is appropriately protected.

Functional Interface ≠ Secure Interface.

## Dependency Security Correspondence

## SSS shall preserve security-relevant relationships with software

dependencies.

Relevant considerations include:

• Known vulnerabilities

• Affected component versions

• Transitive dependencies

• Dependency privileges

• Reachable vulnerable functionality

• Security update availability

• Relevant integration behavior

SDS governs dependency identity, origin, suitability, and continuity.

SSS governs the security consequences of those dependencies.

Identified Dependency ≠ Security-Verified Dependency.

## Configuration Security

## SSS shall assess whether the relevant software configuration supports

applicable security conditions.

Examples include:

• Default credentials

• Exposed debug services

• Excessive privileges

• Disabled protective mechanisms

• Insecure network settings

• Weak cryptographic configuration

SCFS governs configuration identity and integrity.

SSS governs security characteristics of the configuration.

Configuration Integrity ≠ Secure Configuration.

## Software Supply-Chain Security Evidence

## SSS shall recognize security-relevant evidence concerning software

components, artifacts, and build processes.

Evidence may include:

• Component provenance

• Artifact integrity

• Build references

• Dependency vulnerability findings

• Signing records

• Compromise indicators

Provenance and integrity evidence are important but do not establish the absence of vulnerabilities.

Verified Software Origin ≠ Verified Software Security.

## Secrets Handling Security

SSS shall assess software handling of sensitive secrets.

Secrets may include API keys, access tokens, signing credentials, cryptographic keys, and service passwords.

## Relevant conditions include storage, access, transmission, rotation,

revocation, and exposure.

The presence of a secrets-management system does not independently demonstrate secure handling.

Secrets Management Tool ≠ Demonstrated Secrets Protection.

## Security Testing Governance

## SSS shall govern the security-specific purpose, scope, suitability, and

interpretation of software security testing.

Testing methods may include:

• Static analysis

• Dynamic analysis

• Dependency scanning

• Configuration assessment

• Penetration testing

• Fuzz testing

• Security regression testing

• Targeted vulnerability verification

Security testing must remain connected to applicable requirements and threats.

Security Testing Completed ≠ Software Security Demonstrated.

## Security Testing Scope

SSS shall identify the boundaries of security testing.

Scope may include artifact versions, interfaces, threat classes, privilege conditions, configurations, environments, and exclusions.

## A test result shall not be generalized beyond the conditions actually

assessed.

Tested Attack Surface ≠ Complete Attack Surface.

## Security Assessment Method Suitability

SSS shall assess whether a security testing method is suitable for the claim it is intended to support.

Static analysis may identify potentially insecure code patterns without demonstrating runtime exploitability.

Dependency scanning may identify vulnerable versions without establishing that vulnerable functionality is reachable.

Penetration testing may demonstrate resistance to selected attack paths without establishing resistance to every possible attack.

Security Test Execution ≠ Sufficient Security Verification.

## Security Testing Coverage

## SSS shall evaluate security testing coverage against relevant requirements,

threats, attack surfaces, and operating conditions.

Coverage may concern:

• Security properties

• Threat scenarios

• Privileged operations

• Exposed interfaces

• Critical software functions

• Relevant configurations

Numerical coverage alone shall not establish adequate security assurance.

High Security Test Coverage ≠ Proof of Software Security.

## Security Evidence Governance

SSS shall preserve competent evidence supporting security determinations.

Evidence may include:

• Security requirements

• Threat models

• Control verification

• Security test records

• Vulnerability findings

• Remediation evidence

• Configuration references

• Security acceptance decisions

Evidence must remain associated with the software artifact and security condition assessed.

Security Evidence Present ≠ Security Evidence Sufficient.

## Security Evidence Competence

SSS shall assess whether evidence is suitable for the security conclusion being made.

## Competence depends on relevant artifact identity, configuration, threat

conditions, testing method, assessment scope, and known limitations.

A successful security scan against a different artifact is insufficient to establish security of the intended artifact.

Successful Scan ≠ Competent Security Evidence.

## Security Evidence Integrity

SSS shall preserve material security evidence integrity.

Relevant conditions may include:

• Evidence source

• Assessment identity

• Artifact correspondence

• Record integrity

• Temporal relevance

• Material contradictions

• Preservation of historical findings

An intact report may still support an invalid or overly broad conclusion.

Intact Security Report ≠ Valid Security Determination.

## Vulnerability Identification

## SSS shall establish identifiable references for potential software

vulnerabilities.

Findings may originate from testing, scanning, research disclosures, operational observations, incident investigation, human review, or automated analysis.

## A reported vulnerability shall remain distinguishable from a confirmed

vulnerability.

Vulnerability Report ≠ Confirmed Vulnerability.

## Vulnerability Confirmation

## SSS shall establish the conditions under which available evidence supports

recognition of a vulnerability in the relevant software.

Confirmation may require:

• Affected artifact correspondence

• Relevant vulnerability conditions

• Applicable configuration

• Technical evidence

• Exposure assessment

• Reproducible behavior where feasible

A vulnerable dependency version does not automatically establish exploitable application behavior.

Known Vulnerable Component ≠ Demonstrated Exploitable Application.

## Vulnerability Classification

SSS shall classify material vulnerabilities by technical characteristics.

Categories may include injection, authentication weaknesses, authorization weaknesses, data exposure, memory safety failures, cryptographic weaknesses, insecure configuration, and dependency vulnerabilities.

Classification shall remain distinct from severity.

Vulnerability Type ≠ Vulnerability Severity.

## Vulnerability Severity

## SSS shall assess the potential technical significance of confirmed

vulnerabilities.

Severity may depend on affected security properties, required privileges, attack complexity, and potential consequences.

Severity alone shall not determine treatment priority.

Vulnerability Severity ≠ Remediation Priority.

## Vulnerability Exploitability

SSS shall assess the conditions under which a vulnerability may be exploited.

Relevant considerations include:

• Reachability

• Required privileges

• Attack complexity

• Configuration

• Available mitigations

• Dependency behavior

Potential exploitability shall not be confused with observed exploitation.

Potential Exploitability ≠ Observed Exploitation.

## Vulnerability Exposure

## SSS shall evaluate the extent to which relevant vulnerability conditions

are accessible.

Exposure may depend on network accessibility, enabled functionality, user privileges, deployment scope, and integration conditions.

A high-severity vulnerability may have limited exposure.

A lower-severity vulnerability may have extensive exposure.

Vulnerability Severity ≠ Actual Exposure.

## Vulnerability Impact

## SSS shall assess actual or reasonably supported consequences of material

vulnerabilities.

Consequences may include unauthorized access, data disclosure, service disruption, data modification, privilege escalation, or software compromise.

Potential impact shall remain distinguishable from demonstrated harm.

Potential Security Impact ≠ Observed Security Harm.

## Vulnerability Prioritization

SSS shall govern vulnerability treatment priority.

Prioritization may consider:

• Severity

• Exploitability

• Exposure

• Protected assets

• Operational consequences

• Available mitigations

• Remediation feasibility

• Relevant dependencies

A single numerical score shall not substitute for contextual assessment.

Severity Score ≠ Complete Security Risk Determination.

## Vulnerability Remediation

SSS shall preserve references to vulnerability remediation actions.

Actions may include code correction, dependency updates, configuration changes, stronger controls, interface restrictions, removal of vulnerable functionality, or compensating measures.

## Implementation of a corrective action shall remain distinguishable from

verified remediation.

Remediation Implemented ≠ Vulnerability Resolved.

## Remediation Authority

## SSS shall preserve relevant authorization references for security

remediation.

Authority may concern source-code changes, dependency updates, configuration modifications, emergency interventions, or production access.

Identification of a vulnerability does not independently authorize changes to production software.

## Authority to Identify a Vulnerability ≠ Authority to Modify Production

Software.

## Remediation Verification

SSS shall require competent evidence before material remediation is recognized as verified.

Verification may include:

• Reassessment of the original vulnerability

• Corrected artifact identification

• Configuration correspondence

• Relevant security testing

• Verification of expected protective behavior

• Assessment of remaining exposure

## Closing a vulnerability ticket shall not independently establish verified

resolution.

Vulnerability Closure ≠ Verified Vulnerability Remediation.

## Security Regression Governance

## SSS shall recognize security regression risks arising from corrective

actions or software changes.

Regression assessment may concern authentication, authorization, input handling, data protection, dependency interactions, and existing controls.

Correction of one vulnerability does not establish that all other security properties remain unaffected.

One Vulnerability Resolved ≠ Absence of Security Regression.

## Residual Security Risk

## SSS shall identify material security risk remaining after protective

controls, assessment, and remediation.

Residual risk may arise from:

• Known unresolved vulnerabilities

• Incomplete control verification

• Unassessed attack surfaces

• Operational exposure

• Unsupported configurations

• Incomplete remediation

• Threat uncertainty

Residual risk identification shall remain separate from risk acceptance.

Residual Risk Identification ≠ Residual Risk Acceptance.

## Security Exception Governance

## SSS shall recognize explicitly authorized exceptions to security

requirements or treatment expectations.

Exceptions may concern deferred remediation, temporary compensating controls, restricted use, or time-limited exposure.

Each material exception should preserve:

• Exception identity

• Scope

• Justification

• Authority

• Conditions

• Duration

• Reassessment triggers

An authorized exception does not establish that the underlying security requirement has been satisfied.

Authorized Security Exception ≠ Security Conformity.

## Security Conformity Governance

## SSS shall establish supported determinations of conformity with applicable

security requirements.

Possible outcomes include:

## Conforming: Competent evidence supports satisfaction of the applicable

security conditions.

Partially Conforming: Some applicable conditions are supported while others remain unmet or unresolved.

## Nonconforming: Competent evidence demonstrates failure to satisfy

applicable conditions.

Inconclusive: Available evidence is insufficient for determination.

Not Assessed: Relevant conditions have not been evaluated.

Conformity shall remain distinct from acceptance of residual risk.

Security Conformity ≠ Security Risk Acceptance.

## Security Acceptance Authority

## SSS shall preserve the legitimate authority under which residual security

conditions may be accepted.

Authority may derive from organizational governance, delegated responsibility, contractual arrangements, or applicable institutional procedures.

Technical security expertise alone does not establish acceptance authority.

Security Assessment Competence ≠ Security Acceptance Authority.

## Security Risk Acceptance

SSS shall govern the security-specific basis for accepting residual risk.

A competent acceptance decision should identify:

• Software artifact and configuration

• Applicable security requirements

• Relevant vulnerabilities

• Residual risk

• Supporting evidence

• Acceptance authority

• Decision scope

• Conditions and restrictions

• Validity boundaries

• Reassessment triggers

Security acceptance shall not be represented as proof of invulnerability.

Security Acceptance ≠ Proof of Software Invulnerability.

## Conditional Security Acceptance

SSS shall recognize explicitly bounded acceptance where competent authority permits it.

Conditions may include restricted use, limited deployment context, compensating controls, remediation deadlines, additional monitoring, or mandatory reassessment.

Conditional acceptance does not establish complete conformity.

Conditional Security Acceptance ≠ Complete Security Conformity.

## Security Uncertainty

SSS shall preserve uncertainty affecting material security conclusions.

Uncertainty may arise from incomplete threat information, unverified controls, limited testing, unknown exploitability, incomplete artifact correspondence, or changing operating conditions.

Uncertainty shall not be silently converted into security assurance.

Unresolved Security Evidence ≠ Demonstrated Security.

## Security Conflict

A Security Conflict exists where material security evidence, findings, or decisions are incompatible.

## Examples include contradictory vulnerability assessments, conflicting

control verification results, incompatible artifact references, or inconsistent acceptance decisions.

SSS shall preserve the conflict and supporting evidence.

## Conflicting Security Evidence ≠ Permission to Select the Preferred

Conclusion.

## Security Evidence Gap

A Security Evidence Gap exists where material information required for competent security assessment remains unavailable or insufficient.

## Examples include missing control verification, incomplete security testing,

unknown affected artifact, missing remediation evidence, or unresolved dependency exposure.

A gap does not automatically prove either security or insecurity.

Security Evidence Gap ≠ Proof of Security Sufficiency or Failure.

## Critical Security Gap

A Critical Security Gap exists where missing or unresolved evidence prevents competent determination of a materially consequential software security condition.

Examples include:

• A privileged function without verified authorization controls

• A critical vulnerability with unverified remediation

• An exposed interface with unresolved authentication behavior

• A release candidate tested under a different artifact identity

• An autonomous agent with production modification capabilities but

unresolved permission boundaries.

## A critical gap prevents unsupported positive security determinations for

the affected scope.

It does not independently establish that an attack occurred.

Critical Security Gap ≠ Confirmed Security Incident.

## Security Continuity

## SSS shall preserve security relationships across software versions,

configurations, dependencies, environments, and lifecycle transitions.

Continuity may include:

• Security requirements

• Threat models

• Control verification

• Vulnerability history

• Remediation evidence

• Residual risk

• Security exceptions

• Acceptance decisions

• Reassessment records

## Historical security conclusions shall not automatically remain valid after

material changes.

Historical Security Assessment ≠ Current Security Assurance.

## Security Reassessment

SSS shall support reassessment when material security conditions change.

Triggers may include:

• Newly discovered vulnerabilities

• Dependency updates

• Software modifications

• Configuration changes

• New attack surfaces

• Changed threat exposure

• New operational environments

• Compromised credentials

• Invalidated controls

• Expired exceptions

• Contradictory evidence

Reassessment shall preserve relevant historical records.

Security Reassessment ≠ Historical Evidence Erasure.

## Software Security Governance States

SSS shall support identifiable governance states, including:

Requirements-Pending: Material security requirements remain undefined.

## Exposure-Assessment-Pending: Relevant threat exposure has not been

sufficiently assessed.

Controls-Defined: Applicable controls have been identified.

Controls-Implemented: Relevant controls have been implemented.

Control-Verification-Pending: Effectiveness remains unverified.

Assessment-Active: Security assessment is underway.

Evidence-Limited: Material evidence deficiencies remain.

Vulnerability-Detected: A potential vulnerability has been identified.

## Vulnerability-Confirmed: Competent evidence supports vulnerability

recognition.

Remediation-Pending: Required treatment remains outstanding.

Remediation-Implemented: A corrective action has been implemented.

## Remediation-Verification-Pending: Remediation effectiveness remains

unverified.

Remediation-Verified: Competent evidence supports remediation within the assessed scope.

Security-Conforming: Applicable security conditions are sufficiently supported.

Partially-Conforming: Only part of the applicable conditions is sufficiently supported.

## Security-Nonconforming: Competent evidence demonstrates failure to

satisfy applicable requirements.

## Security-Accepted: Competent authority has accepted the assessed security

state.

Conditionally-Accepted: Acceptance is subject to explicit conditions.

## Acceptance-Suspended: Reliance on a previous acceptance has been

suspended.

Acceptance-Withdrawn: A previous acceptance has been withdrawn.

Conflict-Detected: Material security evidence is incompatible.

## Critical-Gap: A materially consequential security condition lacks

sufficient evidence.

Reassessment-Required: Material changes require renewed evaluation.

Unresolved: Available information does not support a sufficient determination.

## Control, verification, conformity, and acceptance states shall remain

distinguishable.

## Security Control State ≠ Security Verification State ≠ Security Acceptance

State.

## Governed Software Security State

The principal output of SSS is:

## Governed Software Security State

The state shall preserve sufficient information to identify:

Security Governance Object: The governed security subject.

Software Identity: The relevant software object.

Artifact Identity: The assessed artifact and version.

Configuration: The applicable configuration.

Security Purpose: The protective objectives.

Security Scope: The assessment boundaries.

Security Requirements: The applicable protective conditions.

Security Assumptions: Material assumptions.

Protected Assets: The relevant assets requiring protection.

Security Properties: The required security characteristics.

Trust Boundaries: The relevant trust relationships.

Threat Conditions: Identified threats.

Threat Applicability: The relevant threat context.

Threat Model: The applicable model and limitations.

Attack Surfaces: The identified attack surfaces.

Security Exposure: The assessed exposure conditions.

Security Controls: The applicable protective mechanisms.

Control Applicability: The suitability of selected controls.

Control Implementation: The implemented conditions.

Control Effectiveness: The verified protective behavior.

## Security Architecture Correspondence: The relationship to implemented

software.

Authentication Security: Relevant identity-verification conditions.

Authorization Security: Relevant permission-enforcement conditions.

Data Protection: The applicable protection conditions.

Input and Output Security: The relevant input and output protections.

Interface Security: The assessed interface protections.

Dependency Security: Relevant dependency-related conditions.

Configuration Security: Relevant security configuration conditions.

Supply-Chain Evidence: Relevant origin and integrity references.

Secrets Handling: Relevant secrets protection conditions.

Security Testing: The performed assessments.

Testing Scope: The assessment boundaries.

Testing Method Suitability: The competence of selected methods.

Testing Coverage: The relevant coverage.

Security Evidence: The supporting records.

Evidence Competence: The suitability of the evidence.

Evidence Integrity: The integrity of supporting records.

Vulnerability Identification: Relevant vulnerability findings.

Vulnerability Confirmation: The recognized vulnerability status.

Vulnerability Classification: The vulnerability type.

Vulnerability Severity: The assessed significance.

Vulnerability Exploitability: The supported exploitation conditions.

Vulnerability Exposure: The relevant accessibility conditions.

Vulnerability Impact: The supported consequences.

Vulnerability Priority: The treatment priority.

Remediation: The relevant corrective actions.

Remediation Authority: The applicable authorization references.

Remediation Verification: The supporting verification evidence.

Security Regression: The relevant regression assessment.

Residual Security Risk: The remaining security limitations.

Security Exceptions: Authorized deviations and their conditions.

Security Conformity: The supported conformity determination.

Security Acceptance Authority: The competent decision authority.

Security Acceptance: The recognized acceptance decision.

Conditional Acceptance: The applicable restrictions.

Security Uncertainty: Material unresolved conditions.

Security Conflicts: Contradictory findings.

Security Evidence Gaps: Missing material evidence.

## Critical Security Gaps: Material unresolved conditions requiring

competent attention.

Security Continuity: The preserved lifecycle relationships.

Security Reassessment: The relevant reassessment history.

Security Governance State: The recognized security condition.

## Downstream References: Information required for competent release,

deployment, change, and lifecycle control governance.

## The state shall not independently establish complete software correctness,

release authorization, deployment authorization, legal compliance, or legal liability.

## SSS Core Module Architecture

SSS is implemented through five Core Modules. Module 1 — SRQM — Security Requirements Module

## Governed Mechanism: Identification, applicability, specification,

protective objectives, security properties, authorization references, and traceability of software security requirements.

## Principal Output: Governed Software Security Requirements State

## Foundational Distinction: Defined Security Requirements ≠ Demonstrated

Security.

SRQM establishes which software security conditions must be governed.

It does not independently determine whether those conditions have been satisfied. Module 2 — SEAM — Security Exposure Assessment Module

## Governed Mechanism: Protected assets, threat identification, threat

applicability, trust boundaries, attack surfaces, exposure conditions, threat modeling, and contextual security exposure.

## Principal Output: Governed Software Security Exposure State

## Foundational Distinction: Identified Threat ≠ Demonstrated Software

Exposure.

SEAM establishes the relevant security exposure conditions.

It does not independently establish control effectiveness or vulnerability remediation. Module 3 — STCM — Security Testing & Control Module

## Governed Mechanism: Security control applicability, implementation

correspondence, effectiveness verification, security testing, coverage, assessment method suitability, and supporting evidence.

## Principal Output: Governed Software Security Testing & Control State

## Foundational Distinction: Implemented Security Control ≠ Verified

Security Control Effectiveness.

## STCM establishes the evidentiary relationship between protective controls,

threat conditions, and observed security behavior.

It does not independently accept residual security risk. Module 4 — VMM — Vulnerability Management Module

## Governed Mechanism: Vulnerability identification, confirmation,

classification, severity, exploitability, exposure, impact, prioritization, remediation references, remediation verification, and continuity.

## Principal Output: Governed Software Vulnerability State

## Foundational Distinction: Vulnerability Closure ≠ Verified Vulnerability

Remediation.

## VMM establishes the governance conditions under which software

vulnerabilities are recognized, assessed, treated, and verified.

It does not independently establish complete software security. Module 5 — SRCM — Security Risk & Conformity Module

## Governed Mechanism: Security conformity assessment, residual risk

evaluation, security exceptions, acceptance eligibility, acceptance authority, conditional acceptance, evidence sufficiency, acceptance validity, and reassessment.

## Principal Output: Governed Software Security Risk & Conformity State

## Foundational Distinction: Security Acceptance ≠ Proof of Software

Invulnerability.

## SRCM establishes the conditions under which software security conformity

and residual security risk acceptance can be competently determined within explicit boundaries.

It does not independently authorize software release or deployment.

## SSS Module Governance Flow

The canonical module governance flow is:

## Security Requirements

↓

## Security Exposure Assessment

↓

## Security Testing & Control

↓

## Vulnerability Management

↓

## Security Risk & Conformity

↓

## Governed Software Security State

## The flow represents governance dependencies rather than a mandatory

one-pass technical workflow.

New vulnerabilities may require revised exposure assessment.

Changes in threat conditions may require revised security requirements.

Failed control verification may require changes to protective mechanisms.

Remediation may require renewed security testing.

Residual risk findings may require additional controls, restrictions, or reassessment.

The modules therefore form an iterative governance system.

## Governance Requirements

A conforming SSS implementation shall satisfy the following requirements.

## SSS-R01 — Security Object Identity: Material security determinations

shall reference identifiable software objects.

## SSS-R02 — Security Purpose: Relevant protective objectives shall remain

explicit.

## SSS-R03 — Security Scope: Security claims shall preserve their assessment

boundaries.

## SSS-R04 — Artifact Correspondence: Security evidence shall remain

connected to the assessed artifact.

## SSS-R05 — Configuration Correspondence: Security determinations shall

preserve relevant configuration conditions.

## SSS-R06 — Security Requirements: Material security requirements shall

remain identifiable.

## SSS-R07 — Requirement Applicability: Requirements shall be assessed

against relevant software context.

## SSS-R08 — Requirement Traceability: Requirements shall remain traceable

to controls, verification, and evidence.

## SSS-R09 — Security Assumptions: Material assumptions shall remain

explicit.

## SSS-R10 — Protected Assets: Relevant protected assets shall remain

identifiable.

## SSS-R11 — Security Properties: Required security properties shall remain

identifiable.

## SSS-R12 — Trust Boundaries: Material trust boundaries shall remain

explicit.

## SSS-R13 — Threat Applicability: Threat conditions shall be assessed

against relevant software exposure.

SSS-R14 — Attack Surface: Relevant attack surfaces shall be identified.

## SSS-R15 — Exposure Assessment: Exposure shall remain distinguishable from

theoretical threat identification.

## SSS-R16 — Control Applicability: Protective controls shall be assessed

for suitability.

## SSS-R17 — Control Implementation: Implemented controls shall remain

identifiable.

## SSS-R18 — Control Verification: Control implementation shall not

substitute for demonstrated effectiveness.

## SSS-R19 — Authentication and Authorization: Material identity and

permission enforcement shall remain assessable.

## SSS-R20 — Dependency Security: Relevant dependency security conditions

shall remain traceable.

## SSS-R21 — Configuration Security: Security determinations shall remain

connected to relevant configuration conditions.

## SSS-R22 — Security Testing: Security testing shall address applicable

security conditions.

## SSS-R23 — Assessment Method Suitability: Methods shall be suitable for

the conclusions they support.

## SSS-R24 — Security Coverage: Material coverage limitations shall remain

explicit.

## SSS-R25 — Evidence Competence: Security determinations shall rely on

competent evidence.

## SSS-R26 — Evidence Integrity: Material evidence shall preserve relevant

integrity conditions.

## SSS-R27 — Vulnerability Recognition: Reported and confirmed

vulnerabilities shall remain distinguishable.

## SSS-R28 — Vulnerability Assessment: Severity, exploitability, exposure,

impact, and treatment priority shall remain distinguishable.

## SSS-R29 — Remediation Traceability: Corrective actions shall remain

connected to relevant vulnerabilities and artifacts.

## SSS-R30 — Remediation Verification: Implemented remediation shall not be

treated as verified resolution.

## SSS-R31 — Security Regression: Material regression conditions shall be

assessed where applicable.

## SSS-R32 — Residual Risk: Material remaining security risk shall remain

explicit.

## SSS-R33 — Security Exceptions: Exceptions shall preserve authority,

scope, conditions, and validity.

## SSS-R34 — Security Conformity: Security conformity shall remain

distinguishable from risk acceptance.

## SSS-R35 — Acceptance Authority: Material acceptance decisions shall have

identifiable legitimate authority.

## SSS-R36 — Conditional Acceptance: Acceptance restrictions shall remain

explicit.

## SSS-R37 — Security Uncertainty: Material uncertainty shall not be

converted into unsupported assurance.

SSS-R38 — Security Conflicts: Contradictory evidence shall remain visible.

## SSS-R39 — Evidence Gaps: Missing evidence shall not be treated as

demonstrated security.

## SSS-R40 — Critical Security Gaps: Material unresolved conditions shall

prevent unsupported positive determinations for the affected scope.

## SSS-R41 — Security Continuity: Security history shall remain traceable

across lifecycle changes.

## SSS-R42 — Security Reassessment: Material changes shall trigger competent

reassessment where applicable.

## SSS-R43 — Security Governance State: The implementation shall produce an

identifiable Governed Software Security State.

## SSS-R44 — Downstream Boundary: Security conformity or acceptance shall

not independently authorize release or deployment.

## Governance Logic

The canonical SSS governance logic is:

## Receive Governed Software Requirements Baseline

↓

## Receive Governed Software Creation State

↓

## Receive Governed Software Dependency State

↓

## Receive Governed Software Configuration State

↓

## Receive Governed Software Quality State

↓

## Identify Software Security Governance Object

↓

## Establish Software Artifact and Configuration Correspondence

↓

## Define Security Purpose and Scope

↓

## Identify Applicable Security Requirements

↓

## Identify Protected Assets and Security Properties

↓

## Establish Security Assumptions and Trust Boundaries

↓

## Identify Relevant Threat Conditions

↓

## Assess Threat Applicability

↓

## Identify Attack Surfaces

↓

## Assess Security Exposure

↓

## Define Applicable Protective Controls

↓

## Assess Control Implementation

↓

## Verify Control Effectiveness

↓

## Establish Security Testing Scope and Methods

↓

## Evaluate Security Testing Coverage

↓

## Preserve Security Evidence

↓

## Assess Evidence Competence and Integrity

↓

## Identify and Confirm Vulnerabilities

↓

## Assess Vulnerability Severity, Exploitability, Exposure, and Impact

↓

## Determine Vulnerability Treatment Priority

↓

## Identify Remediation Actions and Authority References

↓

## Verify Remediation Effectiveness

↓

## Assess Security Regression

↓

## Determine Security Conformity

↓

## Identify Residual Security Risk

↓

## Assess Security Exceptions

↓

## Identify Security Acceptance Authority

↓

## Determine Security Acceptance and Conditions

↓

## Preserve Security Uncertainty, Conflicts, and Evidence Gaps

↓

## Determine Software Security Governance State

↓

## Establish Governed Software Security State

↓

## Handoff to Competent Release and Deployment Governance

↓

## Preserve Security Continuity

↓

Reassess When Material Security Conditions Change

The principal output is:

## Governed Software Security State

## Governance Outputs

SSS may produce:

**Software Security Governance Object**

**Software Security Purpose Record**

**Software Security Scope Record**

**Artifact Security Correspondence Record**

**Configuration Security Correspondence Record**

**Software Security Requirements State**

**Security Requirement Applicability Record**

**Security Requirement Traceability Record**

**Security Assumption Record**

**Protected Asset Register**

**Security Property Record**

**Trust Boundary Record**

**Threat Identification Record**

**Threat Applicability Assessment**

**Threat Model Reference**

**Attack Surface Register**

**Security Exposure Assessment**

**Security Control Register**

**Security Control Applicability Assessment**

**Security Control Implementation Record**

**Security Control Effectiveness Assessment**

**Security Architecture Correspondence Record**

**Authentication Security Assessment**

**Authorization Security Assessment**

**Data Protection Security Assessment**

**Input and Output Security Assessment**

**Software Interface Security Assessment**

**Dependency Security Correspondence Record**

**Configuration Security Assessment**

**Software Supply-Chain Security Evidence Record**

**Secrets Handling Assessment**

**Security Testing Record**

**Security Testing Scope Record**

**Security Assessment Method Suitability Record**

**Security Testing Coverage Assessment**

**Security Evidence Register**

**Security Evidence Competence Assessment**

**Security Evidence Integrity Assessment**

**Vulnerability Identification Record**

**Vulnerability Confirmation Record**

**Vulnerability Classification Record**

**Vulnerability Severity Assessment**

**Vulnerability Exploitability Assessment**

**Vulnerability Exposure Assessment**

**Vulnerability Impact Assessment**

**Vulnerability Prioritization Record**

## Vulnerability Remediation Record

## Remediation Authority Reference

## Remediation Verification Record

## Security Regression Assessment

## Residual Security Risk Record

## Security Exception Record

## Security Conformity Determination

## Security Acceptance Authority Record

## Security Acceptance Decision

## Conditional Security Acceptance Record

## Security Uncertainty State

## Security Conflict

## Security Evidence Gap

## Critical Security Gap

## Security Continuity Record

## Security Reassessment Record

## Software Security Governance State

## Software Release Security Handoff Record

## Software Deployment Security Handoff Record

The principal output is:

## Governed Software Security State

## Critical Distinctions

SSS preserves the following distinctions:

## Declared Software Security ≠ Demonstrated Software Security

**Security Declaration ≠ Governed Security Object**

**Security Objective ≠ Implemented Security Mechanism**

**Component Security Assessment ≠ System Security Assurance**

**Security Assessment of Version A ≠ Security Assessment of Version B**

**Security Control Presence ≠ Active Security Protection**

**Defined Security Requirement ≠ Demonstrated Security**

**General Security Requirement ≠ Applicable Security Requirement**

**Traceable Security Requirement ≠ Verified Security Requirement**

**Assumed Protection ≠ Demonstrated Protection**

**Protected Asset Importance ≠ Vulnerability Severity**

**Declared Security Property ≠ Verified Security Property**

**Technical Access Across a Boundary ≠ Legitimate Authority**

**Identified Threat ≠ Observed Attack**

**Known Threat Pattern ≠ Demonstrated Exposure**

**Completed Threat Model ≠ Demonstrated Software Security**

**Attack Surface ≠ Confirmed Vulnerability**

**Security Exposure ≠ Demonstrated Exploitation**

**Defined Security Control ≠ Implemented Security Control**

**Recommended Control ≠ Sufficient Protection**

**Implemented Security Control ≠ Verified Security Control**

**Successful Control Execution ≠ Demonstrated Protective Effectiveness**

**Documented Security Architecture ≠ Verified Security Implementation**

**Authenticated Identity ≠ Authorized Action**

**Task Authority ≠ Constituent Action Authority**

**Encrypted Data ≠ Fully Protected Data**

**Functional Input Handling ≠ Secure Input Handling**

**Functional Interface ≠ Secure Interface**

**Identified Dependency ≠ Security-Verified Dependency**

**Configuration Integrity ≠ Secure Configuration**

**Verified Software Origin ≠ Verified Software Security**

**Secrets Management Tool ≠ Demonstrated Secrets Protection**

**Security Testing Completed ≠ Software Security Demonstrated**

**Tested Attack Surface ≠ Complete Attack Surface**

**Security Test Execution ≠ Sufficient Security Verification**

**High Security Test Coverage ≠ Proof of Software Security**

**Security Evidence Present ≠ Security Evidence Sufficient**

**Successful Scan ≠ Competent Security Evidence**

**Intact Security Report ≠ Valid Security Determination**

**Vulnerability Report ≠ Confirmed Vulnerability**

**Known Vulnerable Component ≠ Demonstrated Exploitable Application**

**Vulnerability Type ≠ Vulnerability Severity**

**Vulnerability Severity ≠ Remediation Priority**

**Potential Exploitability ≠ Observed Exploitation**

## Vulnerability Severity ≠ Actual Exposure

## Potential Security Impact ≠ Observed Security Harm

## Severity Score ≠ Complete Security Risk Determination

## Remediation Implemented ≠ Vulnerability Resolved

## Authority to Identify a Vulnerability ≠ Authority to Modify Production

Software

## Vulnerability Closure ≠ Verified Vulnerability Remediation

## One Vulnerability Resolved ≠ Absence of Security Regression

## Residual Risk Identification ≠ Residual Risk Acceptance

## Authorized Security Exception ≠ Security Conformity

## Security Conformity ≠ Security Risk Acceptance

## Security Assessment Competence ≠ Security Acceptance Authority

## Security Acceptance ≠ Proof of Software Invulnerability

## Conditional Security Acceptance ≠ Complete Security Conformity

## Unresolved Security Evidence ≠ Demonstrated Security

## Conflicting Security Evidence ≠ Permission to Select the Preferred

Conclusion

## Security Evidence Gap ≠ Proof of Security Sufficiency or Failure

## Critical Security Gap ≠ Confirmed Security Incident

## Historical Security Assessment ≠ Current Security Assurance

## Security Reassessment ≠ Historical Evidence Erasure

## Security Control State ≠ Security Verification State ≠ Security Acceptance

State

## Security-Conforming ≠ Release-Authorized

## Security-Accepted ≠ Deployment-Authorized

## Cross-Architecture Boundary

## SSS governs software-specific security requirements, exposure assessment,

protective controls, security testing, vulnerability management, remediation verification, residual security risk, and security conformity and acceptance within SLGA™.

It does not independently govern organization-wide cybersecurity strategy, enterprise information security management, physical security, legal compliance certification, incident-response command, or general organizational risk acceptance beyond software security.

Within SLGA™:

Software Requirements Standard (SRS) governs software requirements.

Software Creation Standard (SCS) governs software creation.

Software Dependency Standard (SDS) governs software dependencies.

Software Configuration Standard (SCFS) governs software configuration.

Software Quality Standard (SQS) governs software quality.

Software Security Standard (SSS) governs software security.

Software Release Standard (SRLS) governs software release.

Software Deployment Standard (SDPS) governs software deployment.

## Software Change Management Standard (SCMS) governs software lifecycle

changes.

Software Retirement Standard (SRTS) governs software retirement.

## Software Lifecycle Control Standard (SLCS) governs cross-cutting

lifecycle control.

Relevant interoperating OOF® architectures include:

## GOA™ — Governance Architecture: General authority, delegation, and

governance boundaries.

## AGA™ — Accountability Governance Architecture: Relevant responsibility

and accountability relationships.

## ORA™ — Operational Reality Architecture: Assessment of actual operating

conditions and evidence correspondence.

## ASGA™ — Autonomous Systems Governance Architecture: Relevant autonomous

system behavior and control boundaries.

## AGI™ — Agent Governance Infrastructure: Agent permissions, delegated

authority, tool-path governance, and constituent action authorization.

## OBIDENITY® — Origin-Bound Identity Governance Architecture®: Relevant

identity, origin, ownership, provenance, and continuity relationships.

## SSS governs the software security conditions associated with these

relationships without replacing their competent architectural domains.

## Canonical Boundary

SSS begins when a software object requires the establishment, assessment, verification, treatment, acceptance, or reassessment of software security conditions.

SSS governs:

• Software security identity

• Security purpose and scope

• Artifact and configuration correspondence

• Security requirements

• Security assumptions

• Protected assets

• Security properties

• Trust boundaries

• Threat identification and applicability

• Threat modeling

• Attack surfaces

• Security exposure

• Protective controls

• Control applicability and implementation

• Control effectiveness

• Security architecture correspondence

• Authentication and authorization security

• Data and interface protection

• Dependency and configuration security

• Software supply-chain security evidence

• Secrets handling

• Security testing and coverage

• Security evidence competence and integrity

• Vulnerability identification and confirmation

• Vulnerability classification, severity, exploitability, exposure, and

• Vulnerability prioritization

• Remediation and remediation verification

• Security regression

• Residual security risk

• Security exceptions

• Security conformity

• Security acceptance authority

• Conditional acceptance

• Security uncertainty and conflicts

• Security evidence gaps

• Critical security gaps

• Security continuity and reassessment

• Software security governance states

impact.

SSS does not independently determine:

• Enterprise cybersecurity strategy

• Organization-wide information security governance

• Physical security

• Software creation authorization

• Dependency origin validity

• Configuration baseline integrity

• Complete software quality

• General software change authorization

• Incident response command

• Release authorization

• Deployment authorization

• Legal compliance

• Legal liability

SSS ends when material software security conditions have been represented as a:

## Governed Software Security State

## with sufficient software identity, security requirements, threat and

exposure context, protective control evidence, testing evidence, vulnerability and remediation states, residual risk, acceptance conditions, uncertainty, and lifecycle continuity information for competent downstream governance.

## Canonical Closing Statement

## Software security cannot be established merely by implementing protective

controls, completing security scans, closing vulnerability tickets, or declaring that a system is secure.

A security requirement may be documented without being satisfied.

A control may be implemented without providing effective protection.

A vulnerability may remain undetected despite extensive testing.

A reported remediation may remain unverified in the executable artifact.

A component may appear secure in isolation while becoming exposed through integration, configuration, dependencies, or operational conditions.

And a previously accepted security state may become insufficient when threats, vulnerabilities, software, or operating conditions change.

Declared Software Security ≠ Demonstrated Software Security.

Implemented Security Control ≠ Verified Security Control Effectiveness.

Security Testing Completed ≠ Software Security Demonstrated.

Vulnerability Closure ≠ Verified Vulnerability Remediation.

Security Conformity ≠ Security Risk Acceptance.

Security Acceptance ≠ Proof of Software Invulnerability.

These distinctions are especially important as AI coding assistants and autonomous agents increasingly generate software, select dependencies, modify configurations, execute security tests, and propose remediation.

An autonomous system may produce convincing security evidence without demonstrating that the evidence is sufficient.

It may identify a vulnerability without possessing authority to modify production software.

It may report successful remediation without verifying the corrected artifact.

And it may recommend acceptance without possessing legitimate authority to accept residual security risk.

The actor may change. The requirement for legitimate authority does not.

## SSS therefore governs the evidentiary relationship between software

security requirements, actual exposure, protective controls, observed behavior, vulnerabilities, remediation, residual risk, and security acceptance.

It does not promise absolute security.

## It establishes the methodological conditions under which security claims

may be competently assessed, appropriately bounded, and preserved throughout the software lifecycle.

## SSS transforms software security from a collection of controls, scans,

vulnerability records, and compliance declarations into a governed, evidence-supported security state — establishing which protective conditions were required, which threats were relevant, which controls were verified, which vulnerabilities remain, what residual risk exists, and under whose legitimate authority that risk has been accepted.

Software security is not governed because a system has been declared secure. It is governed when its requirements, exposure, controls, verification evidence, vulnerability conditions, residual risks, acceptance boundaries, and lifecycle continuity remain sufficiently established for competent security governance.

## Module Architecture

→ VMM — Vulnerability Management Module

→ SRCM — Security Risk & Conformity Module

## Related Documents

→ About Software Security Standard (SSS)